states, actions, and rewards from an initial state to the terminal state where we win or lose the game. We denote this episode as

$$\varepsilon = (S_0,A_0,R_1),(S_1,A_1,R_2) \ldots (S_{t-1},A_{t-1},R_t)$$

Each tuple is one time-step of the Gridworld game (or a Markov decision process, more generally). After we've reached the end of the episode at time *t*, we've collected a bunch of historical data on what just happened. Let's say that by chance we hit the goal after just three moves determined by our policy network. Here's what our episode looks like:

$$\varepsilon = (S_0,3,-1),(S_1,1,-1),(S_2,3,+10)$$

We've encoded the actions as integers from 0 to 3 (referring to array indices of the action vector) and we've left the states denoted symbolically since they're actually 64-length vectors. What is there to learn from in this episode? Well, we won the game, indicated by the +10 reward in the last tuple, so our actions must have been "good" to some degree. Given the states we were in, we should encourage our policy network to make those actions more likely next time. We want to reinforce those actions that led to a nice positive reward. We will address what happens when our agent loses (receives a terminal reward of –10) later in this section, but in the meantime we will focus on positive reinforcement.

4.2.2. Action reinforcement

We want to make small, smooth updates to our gradients to encourage the network to assign more probability to these winning actions in the future. Let's focus on the last experience in the episode, with state S_2. Remember, we're assuming our policy network produced the action probability distribution [0.25, 0.25, 0.25, 0.25], since it was untrained, and in the last time step we took action 3 (corresponding to element 4 in the action probability array), which resulted in us winning the game with a +10 reward. We want to positively reinforce this action, given state S_2, such that whenever the policy network encounters S_2 or a very similar state, it will be